

Raport Proiect: Sistem de Gestiune a Facultății

Laborator 1 – Sisteme de Gestiune a Bazelor de Date

Nume: Acul Mathyas

Data: 28.02.2026

1. Decizii de Design și Arhitectură

Pentru dezvoltarea acestei aplicații, am optat pentru o **arhitectură stratificată (Layered Architecture)**, separând clar interfața grafică de logica de business și de accesul la baza de date.

Principalele decizii de design au fost:

- **Interfață Single-Window (Master-Detail):** Am luat decizia directă de a include toate entitățile (Profesori, Studenți, Materii) în aceeași fereastră principală. Acest aspect permite observarea ușoară și rapidă a modului în care modificările se propagă în widget-urile aplicației. De asemenea, m-am asigurat că designul și dispunerea butoanelor pentru fiecare entitate sunt simetrice și asemănătoare, pentru a oferi un aspect vizual plăcut și o experiență de utilizare (UX) intuitivă.
- **Reutilizarea codului prin Generics:** Pentru a nu scrie interogări SQL și logică repetitivă de 3 ori (pentru fiecare tabel în parte), am creat clase abstracte și interfețe generice (ex: `Repository<ID, E>`, `DBRepository`, `Service<ID, E>`). Astfel, logica CRUD principală este implementată o singură dată, respectând principiul *DRY (Don't Repeat Yourself)*.
- **Design Pattern-ul Observer:** Controller-ul interfeței grafice acționează ca un *Observer*, iar clasele de *Service* ca *Observables*. Această decizie decuplează complet baza de date de interfață; la orice modificare (Add/Update/Delete), interfața este notificată automat să își facă refresh.

2. Provocări Întâmpinate și Soluții

- **Provocarea 1: Sincronizarea datelor și evitarea codului duplicat.**
 - *Context:* Pe măsură ce adăugam noi entități, riscam ca metodele de reîmprospătare a tabelelor să devină greu de întreținut și foarte repetitive.
 - *Soluția:* Implementarea șabloanelor de proiectare (Design Patterns). Am creat clasa `AbstractController` și funcții generice, iar combinarea acestora cu pattern-ul *Observer* și abstractizarea entităților (clasa de bază `Entity`) m-a ajutat să mențin un *Clean Code* și să scal aplicația foarte ușor.
- **Provocarea 2: Ștergerea entităților părinte și constrângerile bazei de date.**

- *Context:* La încercarea de a șterge un Student sau un Profesor care era deja asociat unei materii, PostgreSQL arunca o eroare de tipul *Foreign Key Constraint Violation*, pentru a preveni apariția înregistrărilor orfane în tabela de legătură.
- *Soluția:* În loc să scriu query-uri multiple de "curățare" în codul Java, am rezolvat problema direct din arhitectura bazei de date, folosind constrângerea `ON DELETE CASCADE` la crearea cheilor străine.

3. Ce am învățat din acest proiect

Acest laborator a reprezentat o oportunitate excelentă de a îmbina cunoștințele de baze de date cu cele de programare orientată pe obiecte. Printre cele mai importante lecții învățate se numără:

- **Managementul timpului și viziunea de ansamblu:** Am învățat cât de important este să privesc proiectul din ansamblu înainte de a scrie prima linie de cod. Analizând arhitectura la început (entități, repository, service), am reușit să mă apuc mult mai eficient de lucru.
- **Importanța planificării vs. Refactorizare:** Chiar dacă am petrecut mai mult timp la început pentru a gândi logica claselor abstracte și a claselor generice, am recuperat acel timp pe parcurs. Faptul că nu am fost nevoit să refac clasele sau să rescriu funcții întregi pentru a păstra aspectul de *clean-code* mi-a dovedit că un design bun de la început previne refactorizările costisitoare de la final.
- **Conectivitatea Low-Level:** Am aprofundat lucrul direct cu JDBC (fără ORM-uri precum Hibernate), înțelegând exact cum se deschid conexiunile, cum funcționează `PreparedStatement` (vital pentru prevenirea atacurilor de tip *SQL Injection*) și cum se mapează manual rezultatele (`ResultSet`) în obiecte Java.